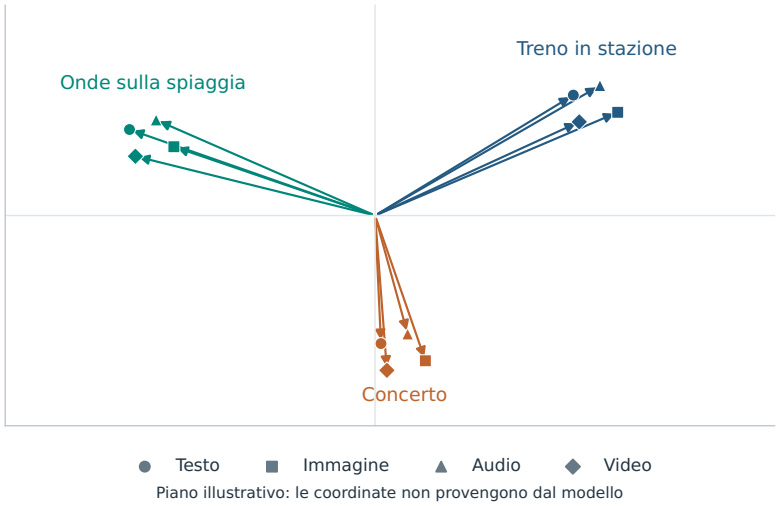
CAPIRE GLI EMBEDDING E LA RICERCA MULTIMODALE

Dagli embedding a EmbeddingGemma 2

Come si calcolano gli embedding e come si usano per confrontare testi, immagini, audio e video

Che cosa sono gli embedding, come vengono appresi e a cosa servono nella ricerca semantica. La seconda parte segue il percorso dei dati in EmbeddingGemma 2: dagli encoder al pooling, fino al Matryoshka Representation Learning.



Testo, immagini, suoni e video possono riferirsi allo stesso contenuto. La ricerca multimodale rende confrontabili le loro rappresentazioni. Schema qualitativo, con coordinate illustrative.

Guida in italiano · Revisione del 29 ottobre 2026

Indice

I · CAPIRE GLI EMBEDDING

1. Che cos’è un embedding 3

2. A cosa servono gli embedding 5

3. Come si addestra un modello di embedding 6

4. Dal testo all’embedding 8

II · EMBEDDINGGEMMA 2

5. EmbeddingGemma 2 e la multimodalità 10

6. Il percorso dei dati dentro il modello 12

7. Contesto, chunking e Matryoshka 17

La prima parte introduce gli embedding senza richiedere conoscenze di programmazione. La seconda descrive EmbeddingGemma 2; ogni formula è accompagnata da una spiegazione. I riferimenti alle fonti sono cliccabili nel testo.

PARTE I · CAPIRE GLI EMBEDDING

1. Che cos'è un embedding

Abbiamo salvato una guida intitolata «Come prolungare l'autonomia del portatile». Qualche mese dopo la cerchiamo scrivendo «la batteria del computer dura poco». Le parole cambiano, ma la richiesta riguarda lo stesso argomento. Gli embedding permettono a un programma di riconoscere collegamenti di questo tipo.

Un **embedding** è una lista ordinata di numeri che rappresenta un contenuto: per esempio una frase, un'immagine o un suono. Un modello la calcola in modo che, confrontando le liste, un programma possa riconoscere somiglianze e collegamenti utili fra i contenuti. Nel nostro esempio, la ricerca sulla batteria dovrebbe risultare più vicina alla guida sull'autonomia che a una ricetta di cucina.

1.1 Che aspetto ha

Un embedding potrebbe cominciare così: `[0.12, -0.34, 0.08, 0.51, …]`. Sono valori inventati per mostrarne l'aspetto. La lista completa può contenerne centinaia. Questa lista ordinata si chiama *vettore*; la sua *dimensione* è il numero di valori che contiene.

EmbeddingGemma 2 produce normalmente embedding di 768 numeri. Una frase breve e un paragrafo possono quindi avere rappresentazioni della stessa dimensione, pur contenendo quantità diverse di testo. (model card)

1.2 Una mappa dei collegamenti

Possiamo immaginare ogni contenuto come un punto su una mappa. Frasi su argomenti collegati occupano zone vicine; contenuti poco collegati tendono a stare più lontani. Per cercare la nostra guida, il programma colloca sulla stessa mappa anche la richiesta «la batteria del computer dura poco» e guarda quali contenuti si trovano nelle vicinanze.

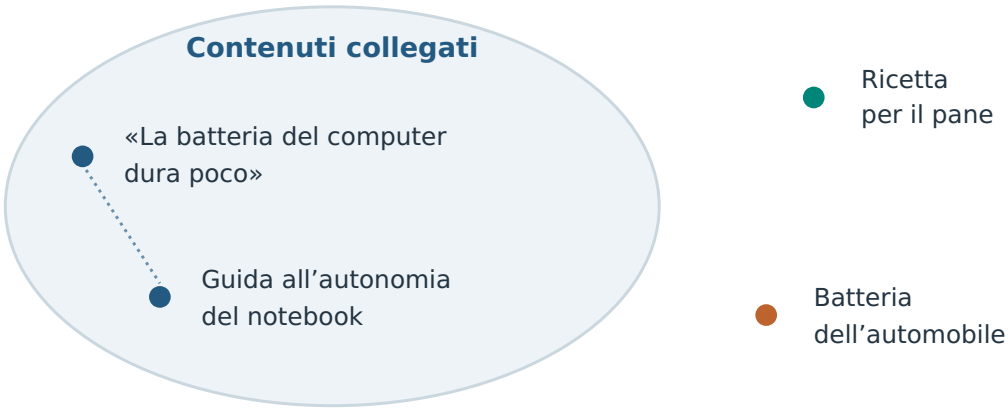


Figura 1. Mappa illustrativa: frasi con parole diverse possono esprimere idee simili. Le coordinate sono scelte per l'esempio.

La mappa è un'immagine mentale. Un embedding reale ha molte più coordinate di quelle che possiamo disegnare su un foglio. Per confrontare due embedding si usano regole numeriche, come la *similarità coseno*: qui basta sapere che restituiscono un punteggio con cui ordinare i contenuti secondo la somiglianza riconosciuta dal modello.

1.3 Che cosa significano quei numeri

Di solito il primo numero non indica «quanto si parla di batterie», né il secondo «quanto si parla di computer». L'informazione è distribuita fra molte coordinate. È il confronto fra i vettori nel loro insieme a far emergere le somiglianze fra i contenuti.

L'embedding è una sintesi: può mettere in evidenza il tema di una pagina senza conservarne ogni dettaglio. Per leggere la guida occorre ancora aprire il testo originale. Inoltre, due modelli diversi possono costruire mappe diverse: per confrontare gli embedding bisogna usare lo stesso modello, o modelli progettati per produrre rappresentazioni compatibili.

1.4 Che cosa ci dice una somiglianza

Un punteggio alto segnala un collegamento secondo il modello. Non è una percentuale di correttezza: una similarità di 0,8 non significa «risposta giusta all'80%». Anche se nell'archivio manca la guida cercata, ci sarà comunque un contenuto più vicino degli altri.

Conta anche il tipo di collegamento. «Il computer si accende» e «Il computer non si accende» condividono quasi tutto l'argomento, ma descrivono situazioni opposte. Negazioni, date e codici di prodotto possono essere decisivi e richiedere controlli specifici. Gli embedding aiutano a individuare i candidati interessanti; la qualità del risultato va verificata sugli esempi reali dell'applicazione.

2. A cosa servono gli embedding

Gli embedding si usano per cercare, organizzare e suggerire contenuti. Dopo averne calcolato le rappresentazioni, il programma può confrontarle anche quando i testi usano parole diverse.

2.1 Trovare qualcosa anche usando parole diverse

Nella **ricerca semantica** possiamo descrivere ciò che cerchiamo con parole nostre. Il testo della ricerca si chiama *query*: «la batteria dura poco» può portare alla guida «Come prolungare l'autonomia del portatile». Un modello multilingue può collegare anche query e documenti in lingue diverse.

Si calcola in anticipo l'embedding di ogni contenuto e lo si salva insieme al riferimento al file. Quando arriva una query, basta calcolarne l'embedding e confrontarlo con quelli già pronti. Il programma ordina i risultati per somiglianza. Questo confronto richiede generalmente molto meno lavoro che elaborare di nuovo tutti i documenti con il modello a ogni ricerca. (Sentence-BERT)

Per un manuale lungo conviene spesso rappresentare separatamente le sezioni: così una procedura di poche righe può emergere anche se il resto del manuale parla d'altro. Il risultato utile è il collegamento alla sezione che ci interessa.

2.2 Organizzare e mettere in relazione

La ricerca è soltanto uno degli usi. Lo stesso principio si presta a compiti diversi:

Uso	Esempio concreto	Come aiutano gli embedding
Raggruppare	Scoprire i temi ricorrenti in mille commenti dei clienti.	Avvicinano commenti simili, anche quando usano parole diverse.
Classificare	Proporre il reparto a cui assegnare una richiesta di assistenza.	Permettono di confrontarla con esempi già classificati o descrizioni delle categorie.
Suggerire	Trovare articoli collegati a quello appena letto.	Aiutano a selezionare contenuti vicini per argomento.
Rilevare quasi duplicati	Individuare due segnalazioni dello stesso problema, scritte in modo diverso.	Segnalano coppie da controllare, anche senza una corrispondenza parola per parola.
Cercare fra media diversi	Ritrovare una foto descrivendo «un treno sotto una pensilina».	Un modello multimodale può rendere confrontabili la descrizione e l'immagine.

Il modello fornisce la rappresentazione; il programma che la usa decide come formare i gruppi, proporre una categoria o mostrare i suggerimenti. Per confrontare testo, immagini e suoni serve un modello addestrato a collegare quelle forme di informazione.

3. Come si addestra un modello di embedding

Durante l'addestramento si presentano al modello esempi di contenuti da collegare e da distinguere. Il modello modifica i propri parametri per produrre embedding che rispettino queste relazioni. Non occorre assegnare a mano il vettore corretto a ogni frase.

3.1 Coppie positive ed esempi negativi

Immaginiamo di addestrare un modello per cercare fra guide pratiche. Gli presentiamo la richiesta «la batteria del portatile dura poco» insieme a tre testi:

- **Un abbinamento utile:** una guida per ridurre i consumi del portatile.
- **Un abbinamento fuori tema:** una ricetta per preparare il pane.
- **Un abbinamento ingannevole:** istruzioni sulla batteria di un'automobile.

L'obiettivo è far risultare la guida al portatile più vicina alla richiesta rispetto agli altri due testi. La guida e la richiesta formano una *coppia positiva*; gli abbinamenti non pertinenti sono esempi *negativi*. Il terzo testo è particolarmente istruttivo: contiene la parola «batteria», ma riguarda un oggetto diverso.

Questo tipo di addestramento si chiama **contrastive learning**, o apprendimento contrastivo. Si può applicare anche a una fotografia e alla sua descrizione, oppure a due frasi dal significato simile. CLIP è un esempio noto di modello addestrato a confrontare immagini e testi. (CLIP)

3.2 La loss e l'aggiornamento dei pesi

All'inizio dell'apprendimento, gli abbinamenti possono essere poco utili. Il programma calcola gli embedding degli esempi e li confronta. Se la richiesta risulta più vicina alla ricetta che alla guida pertinente, una misura dell'errore, chiamata *loss*, segnala che il risultato va corretto.

Il modello contiene numeri regolabili, chiamati **pesi** o **parametri**, che determinano le trasformazioni applicate ai dati. L'algoritmo di addestramento calcola come variare questi pesi per ridurre la loss e li aggiorna, ripetendo il procedimento su molti esempi. I pesi appartengono al modello; l'embedding è il vettore prodotto per un contenuto. Un modello con milioni di parametri può quindi restituire un embedding di poche centinaia di numeri.

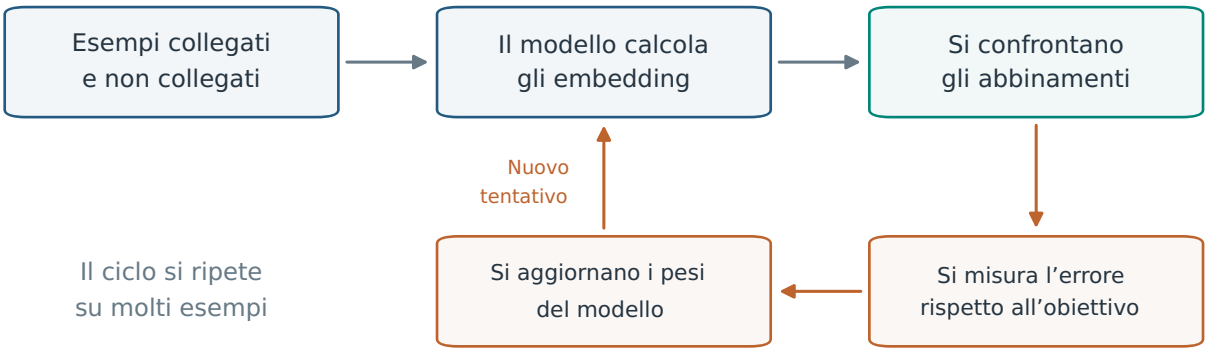


Figura 2. Ciclo di addestramento: la loss guida l'aggiornamento dei pesi, che cambia gli embedding prodotti dal modello.

Cambiando i pesi, cambiano anche gli embedding che il modello calcolerà in seguito. Se l'addestramento funziona, una frase mai incontrata prima, come «devo ricaricare il notebook ogni ora», può risultare vicina alle guide sull'autonomia.

Gli esempi negativi servono a evitare una soluzione inutile: produrre lo stesso vettore per qualsiasi contenuto. In quel caso tutte le coppie risulterebbero simili e la ricerca non distinguerebbe una guida da una ricetta.

3.3 Dati di addestramento e pretraining

Le relazioni possono provenire da raccolte preparate da persone, da domande associate alle loro risposte, da traduzioni o da immagini accompagnate da descrizioni. Questi collegamenti forniscono il segnale con cui imparare. Errori e lacune nei dati possono insegnare associazioni poco utili.

Molti modelli di embedding partono da una rete già addestrata su grandi quantità di dati. Questa prima fase, il *pretraining*, permette di apprendere regolarità del linguaggio e relazioni fra parole. Un addestramento successivo adatta le rappresentazioni al confronto fra frasi o documenti. Se si parte da zero, invece, i pesi vengono inizializzati, spesso con valori casuali, e corretti durante l'addestramento. (Google ML Crash Course; Sentence-BERT)

Gli esempi dipendono dal compito. Per riconoscere parafrasi servono frasi dal significato equivalente; per la ricerca servono anche domande abbinate a testi che rispondono. «Quanto dura la batteria?» e «L'autonomia dichiarata è di otto ore» sono pertinenti senza essere parafrasi.

3.4 Addestramento e inferenza

Quando usiamo un modello già pronto, i pesi sono normalmente fissi: il modello applica ciò che ha imparato a un nuovo contenuto e ne calcola l'embedding. Questa fase si chiama *inferenza*. Aggiungere cento documenti al nostro archivio significa calcolare cento nuove rappresentazioni; di per sé non riaddestra il modello.

4. Dal testo all'embedding

Un **embedder** è un modello che calcola embedding. Molti embedder usano un *Transformer*, una rete neurale che elabora le relazioni fra le parti dell'input. Vediamo come trasforma la frase «la batteria del portatile dura poco» in un vettore.

4.1 La tokenizzazione

Un componente chiamato *tokenizer* divide il testo in **token**: pezzi che possono essere parole intere, parti di parole o segni di punteggiatura. Ogni token riceve un identificatore e viene associato a una prima rappresentazione numerica, attraverso una tabella appresa dal modello.

Il vettore iniziale di un token non descrive ancora il significato dell'intera frase. Il modello riceve anche informazioni sulla posizione dei token, per tener conto del loro ordine.

4.2 I token e il contesto

Il Transformer aggiorna le rappresentazioni attraverso più strati. Un meccanismo centrale è l'**attention**, o attenzione: ogni posizione combina informazioni provenienti dalle altre, con pesi che dipendono dall'input. Nella nostra frase, «portatile» e «dura poco» aiutano a rappresentare «batteria» come fonte di energia. In «suona la batteria nel gruppo», il contesto suggerisce invece lo strumento musicale. (Attention Is All You Need)

Queste trasformazioni dipendono dai pesi appresi. La rappresentazione di un token cambia così in funzione degli altri token presenti nella frase.

EmbeddingGemma 2 usa attenzione *bidirezionale*: le rappresentazioni possono incorporare informazioni sia da ciò che precede sia da ciò che segue nel testo, entro i collegamenti consentiti dai suoi strati. Il dettaglio dell'architettura è descritto nella Parte II. (codice Transformers)

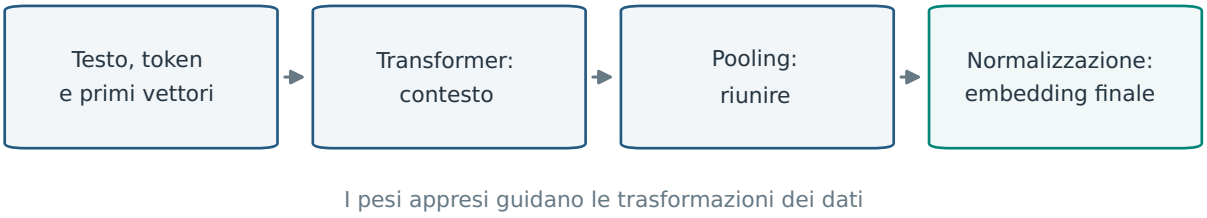


Figura 3. Dal testo all'embedding: tokenizzazione, elaborazione del contesto e pooling.

4.3 Pooling e normalizzazione

Dopo il Transformer resta un vettore per ogni posizione. Il **pooling** li riunisce in un unico embedding. Nel pacchetto ufficiale Sentence Transformers di EmbeddingGemma 2, una trasformazione appresa porta i vettori alla dimensione di uscita; poi si calcola la media delle posizioni valide. (configurazione Sentence Transformers)

Il pooling avviene dopo che il Transformer ha elaborato il contesto. Cambiare l'ordine dei token può quindi cambiare i vettori su cui si calcola la media, e con essi l'embedding finale.

La configurazione ufficiale applica infine una *normalizzazione L2*: il vettore viene diviso per la sua norma, così da avere lunghezza geometrica uno. Questa operazione conserva la direzione del vettore e rende più semplice il confronto, come vedremo nella sezione 6.7.

4.4 Lo stesso principio oltre il testo

Immagini e audio richiedono una preparazione diversa dal testo. Un modello multimodale usa *encoder* dedicati, cioè reti che trasformano immagini o segnali audio in rappresentazioni numeriche. In EmbeddingGemma 2 queste rappresentazioni passano poi attraverso una rete condivisa, detta *backbone*. Per i video si elaborano i frame selezionati. (guida Google)

Il collegamento fra «un treno in stazione» e una fotografia della scena dipende dall'addestramento che allinea le rappresentazioni. Produrre due liste della stessa lunghezza, da solo, non basta a renderle confrontabili.

PARTE II · IL MODELLO

5. EmbeddingGemma 2 e la multimodalità

EmbeddingGemma 2 è un modello di Google DeepMind presentato il 6 ottobre 2026. Deriva dall'architettura Gemma 4 ed è distribuito con licenza Apache 2.0. La sua funzione è produrre rappresentazioni confrontabili di testo, codice, immagini, audio e video. L'output nativo è un vettore di 768 coordinate. La disponibilità dei pesi permette di calcolare queste rappresentazioni sul dispositivo che contiene i dati. (annuncio Google)

EmbeddingGemma 2 è la seconda generazione del modello di embedding. Gemma 2 è invece un modello generativo; Gemini Embedding 2 è un altro prodotto. Il riferimento a Gemma 4 riguarda l'architettura da cui deriva EmbeddingGemma 2.

5.1 Che cosa significa multimodale

Una *modalità* è una forma in cui si presenta l'informazione. Un testo è una sequenza di simboli; un'immagine ha una struttura spaziale; l'audio è un segnale nel tempo; il video unisce una sequenza di immagini ed eventualmente una traccia sonora. In questo modello il codice sorgente rientra nel percorso del testo, mentre i frame video vengono elaborati dal percorso visivo.

Nella ricerca multimodale si confrontano contenuti di tipo diverso: una query testuale può trovare una fotografia o una registrazione audio. Questo richiede embedding in uno **spazio condiviso e allineato**. Due modelli indipendenti possono produrre vettori della stessa dimensione senza che le loro coordinate siano confrontabili.

Una foto della banchina, il rumore di una locomotiva e la frase «un treno in stazione» descrivono aspetti della stessa scena. L'addestramento multimodale permette di avvicinare queste rappresentazioni. Le informazioni disponibili restano diverse: il suono, per esempio, non rivela il colore del treno.

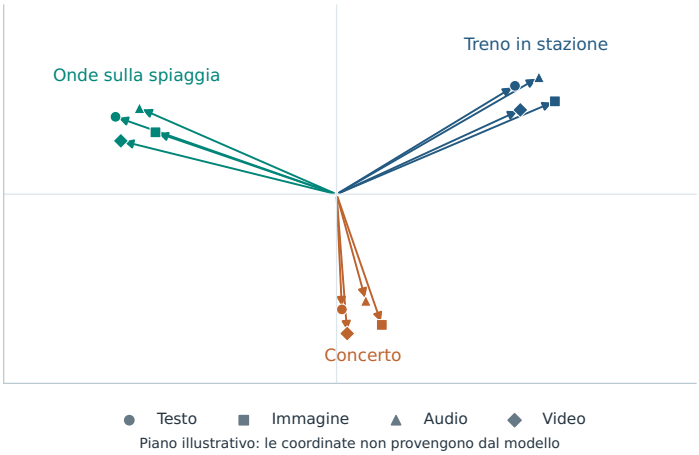


Figura 4. Spazio multimodale illustrativo. Le forme distinguono le modalità, i colori gli argomenti. Le coordinate sono scelte per l'esempio.

5.2 Multimodalità nativa e input interleaved

Un sistema può rendere ricercabile un'immagine generandone prima una descrizione, da passare a un encoder testuale. In questo caso la ricerca dipende da ciò che è stato descritto: i dettagli omessi non entrano nell'embedding.

EmbeddingGemma 2 elabora immagini e audio con encoder dedicati e passa le loro rappresentazioni al backbone condiviso. Può ricevere anche input misti, per esempio un testo accompagnato da due fotografie. (guida Google)

In un input *interleaved*, testo e media si alternano nella stessa sequenza: «vista frontale», una foto; «dettaglio del connettore», un'altra foto. L'ordine aiuta a collegare ciascun testo all'immagine corrispondente. Il modello produce un unico embedding per questa sequenza, confrontabile con gli embedding di altri contenuti. (guida multimodale)

Questo risultato non coincide, in generale, con la media dei vettori ottenuti separatamente dal testo e dalle immagini. Nell'input congiunto le componenti possono interagire durante l'elaborazione. In termini schematici, f_θ(testo, immagine) non è necessariamente uguale a una combinazione lineare di f_θ(testo) e f_θ(immagine).

5.3 Un modello modulare

Il modello completo comprende circa 740 milioni di parametri. La base testuale richiede circa 270 milioni di parametri; l'encoder visivo ne aggiunge circa 170 milioni e quello audio circa 300 milioni. È possibile caricare soltanto gli encoder necessari. La base condivisa rimane coinvolta anche quando l'input è un'immagine o un audio. (guida Google)

Configurazione	Parametri approssimativi	Input gestibili
Base	270 milioni	Testo e codice
Base + visione	440 milioni	Testo, immagini, frame video
Base + audio	570 milioni	Testo e audio
Completa	740 milioni	Testo, immagini, video e audio

Per indicizzare un archivio multimediale possono servire tutti gli encoder. Per calcolare l'embedding di una query testuale basta invece la base. I vettori restano confrontabili se provengono dallo stesso modello e usano le stesse dimensioni e la stessa normalizzazione.

La prima generazione di EmbeddingGemma era dedicata al testo e aveva un contesto di circa 2.000 token. La seconda estende il progetto alle modalità non testuali e porta il contesto documentato a 8.192 token. Il supporto multilingue dichiarato supera le cento lingue; non implica qualità identica in ogni lingua o dominio. (documentazione Google)

6. Il percorso dei dati dentro il modello

Testo, immagini e audio entrano nel modello attraverso percorsi diversi. Gli encoder ne ricavano rappresentazioni che il backbone elabora insieme; il pooling le riunisce infine in un embedding. La figura mostra questi passaggi dall'alto verso il basso.

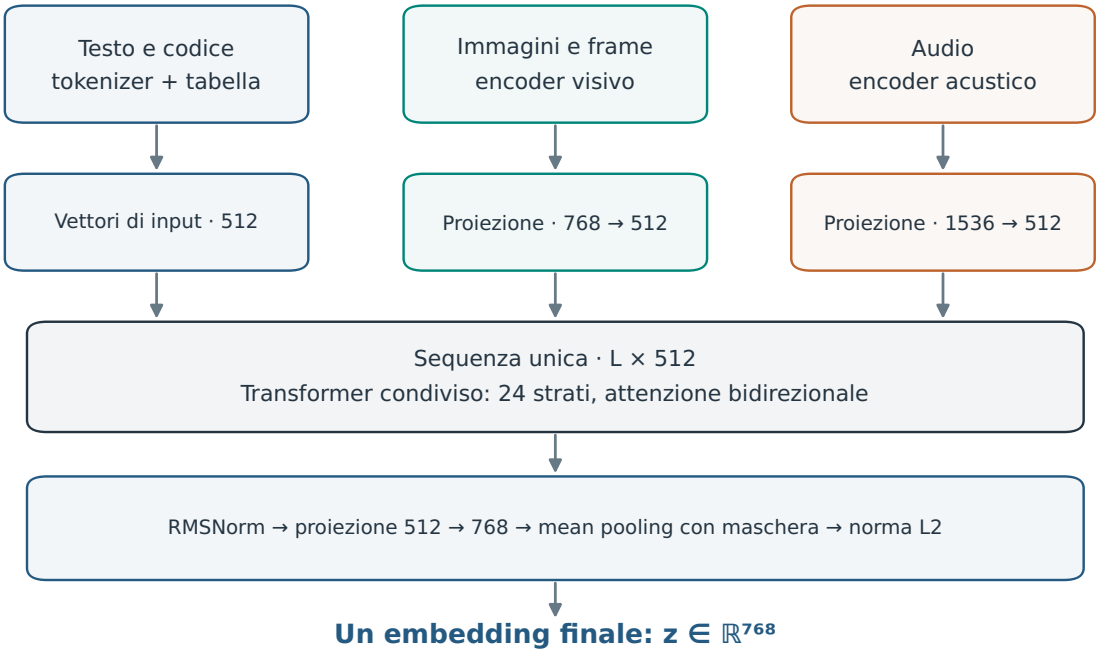


Figura 5. Architettura di EmbeddingGemma 2. Le proiezioni di ingresso adattano le dimensioni degli encoder al backbone. La proiezione finale porta ogni posizione a 768 dimensioni; il pooling produce un unico vettore. (configurazione del modello; codice Transformers; configurazione Sentence Transformers)

6.1 Il testo e il codice

Il tokenizer converte il testo in identificatori interi. Una tabella assegna a ciascun identificatore un vettore iniziale. La configurazione pubblica indica un vocabolario di 262.144 elementi e una dimensione interna del backbone pari a 512. (configurazione del modello) La tokenizzazione di una parola italiana può produrre più token; il conteggio delle parole non coincide quindi con quello dei token.

Il codice sorgente viene trattato come testo: contiene operatori, nomi di funzioni, commenti e struttura sintattica. La query «ritenta una richiesta aumentando l'attesa» può trovare una funzione che implementa un *backoff*, cioè attese progressivamente più lunghe fra i tentativi, anche se la frase non compare nel codice.

6.2 Le immagini

Il *processor* prepara gli input nel formato richiesto dal modello. L'encoder visivo riceve l'immagine preparata, la suddivide in *patch*, piccoli riquadri, e ne elabora il contenuto e la posizione. La configurazione pubblica specifica patch di 16 pixel di lato, 16 strati visivi e una dimensione interna di 768. Il risultato viene ridotto a una sequenza di *soft token* e proiettato nelle 512 dimensioni del backbone. (configurazione del modello)

Un *soft token* è un vettore che rappresenta una parte dell'input e viene inserito nella sequenza del modello. Può contenere informazioni su forme, testo visibile, oggetti e relazioni spaziali, apprese dall'encoder. Il modello non deve prima trasformarlo in una parola o in una trascrizione OCR. Più soft token permettono una rappresentazione visiva più dettagliata, ma richiedono più calcolo.

Per un documento impaginato, l'immagine di una pagina conserva elementi come colonne, grafici e posizione delle etichette. Un'estrazione del solo testo può perdere queste relazioni. Le due rappresentazioni possono essere utili per domande diverse: trovare una pagina con un certo diagramma non è lo stesso compito che trovare una stringa esatta nel testo.

6.3 L'audio

Il processor prepara l'audio come segnale mono a 16 kHz. L'encoder audio lo elabora direttamente, senza richiedere una trascrizione intermedia, e produce una sequenza di rappresentazioni poi adattate alla dimensione del backbone. (guida multimodale)

A 16 kHz, dieci secondi di audio contengono 160.000 campioni. Il processor e l'encoder ne ricavano una sequenza di rappresentazioni molto più corta. Il pooling la riunisce in un unico embedding. Il numero di campioni audio, il numero di posizioni nella sequenza e la dimensione dell'embedding sono quindi grandezze diverse.

La ricerca può riguardare il parlato o un evento sonoro, come un applauso. L'embedding rappresenta il contenuto della registrazione, ma non ne restituisce la trascrizione: per quella serve un sistema di riconoscimento vocale.

6.4 Il video e il tempo

Nel percorso documentato, il video viene campionato in fotogrammi, elaborati dall'encoder visivo. La frequenza predefinita è un frame al secondo e può essere modificata. Il processor espone anche il numero massimo di frame, una strategia per gestire quelli in eccesso e l'inserimento di timestamp. La guida riporta un limite predefinito del processor pari a 32 frame: è distinto dal limite teorico del contesto del modello. (guida multimodale)

Un'azione più breve dell'intervallo di campionamento può non apparire in alcun frame. Per trovare eventi rapidi servono quindi un campionamento più frequente o una selezione mirata dei fotogrammi. Per cercare anche ciò che viene detto, la pipeline deve elaborare separatamente la traccia audio o includerla nell'input multimodale.

6.5 La sequenza condivisa e l'attention

Consideriamo una scheda con la frase «il connettore della batteria» accanto alla foto di un portatile aperto. Il backbone elabora insieme il testo e le rappresentazioni visive, così che le parole possano essere messe in relazione con il connettore nella foto.

Gli adattatori portano tutte le rappresentazioni alla dimensione del backbone: 512 numeri per posizione. Possiamo disporle in una matrice con L righe, una per ogni token o soft token, e 512 colonne. Il numero di righe dipende dall'input. Informazioni di posizione e marker dei media ne descrivono l'ordine e il tipo. Il Transformer elabora questa sequenza:

$$X \in \mathbb{R}^{L \times 512} \quad \longrightarrow \quad H = T_\theta(X) \in \mathbb{R}^{L \times 512}.$$

Come si legge. Il Transformer T, con parametri θ, trasforma la matrice X nella matrice H. La notazione $\mathbb{R}^{L \times 512}$ indica L righe di 512 numeri reali. Il numero di righe e colonne resta uguale; cambiano i valori, che ora incorporano informazioni dalle altre posizioni.

L'**attention** combina informazioni provenienti da posizioni diverse. Per ogni posizione, il modello calcola tre vettori: *query*, *key* e *value*. Il confronto fra la query di una posizione e le key delle altre determina i pesi con cui combinare i value. Qui "query" indica un vettore interno al modello, non la ricerca scritta dall'utente.

I parametri appresi determinano come calcolare query, key e value; i pesi dell'attention cambiano con l'input. La combinazione dei value si scrive così:

$$\mathbf{y}_i = \sum_j \alpha_{ij} \mathbf{v}_j.$$

Come si legge. Per la posizione i, si moltiplica ciascun vettore v_j per il suo peso α_{ij} e si sommano i risultati. Il simbolo Σ indica questa somma; y_i è il vettore ottenuto. I pesi sono non negativi e, per una stessa posizione i, la loro somma è uno.

Per esempio, con pesi 0,6, 0,3 e 0,1, i tre vettori contribuiscono rispettivamente per il 60%, il 30% e il 10% alla somma. Il calcolo si applica a ogni coordinata dei vettori. Questi pesi sono illustrativi: il modello li calcola per l'input ricevuto. I suoi strati ripetono l'operazione insieme ad altre trasformazioni. (Attention Is All You Need)

6.6 Attenzione bidirezionale, locale e globale

L'attention può usare solo le posizioni consentite dalla sua *maschera*. Le regole dipendono dal tipo di modello e dallo strato.

Un tipico LLM generativo autoregressivo produce un token alla volta. Ogni posizione usa ciò che la precede e il proprio stato per prevedere il seguito; le posizioni successive sono escluse. È l'attenzione **causale**. Durante l'addestramento il vincolo vale anche se la frase completa è disponibile: impedisce al modello di consultare la continuazione che deve imparare a prevedere. (Attention Is All You Need)

Con l'attenzione **bidirezionale**, una posizione può usare sia il contesto precedente sia quello successivo. La rappresentazione di «batteria», per esempio, può incorporare «del portatile» e «dura poco», che vengono dopo. È un modo di collegare le posizioni, non una doppia lettura del testo.

Un chatbot autoregressivo può comunque usare tutta la domanda quando comincia a rispondere: l'intero prompt precede i token della risposta. Il vincolo causale riguarda le posizioni successive a quella che sta elaborando.

EmbeddingGemma 2 usa attenzione bidirezionale, alternando strati *locali* e *globali*. L'attenzione locale collega le posizioni vicine; quella globale può collegare tutta la sequenza valida. Uno strato locale può quindi essere bidirezionale senza accedere all'intero input. (codice Transformers)

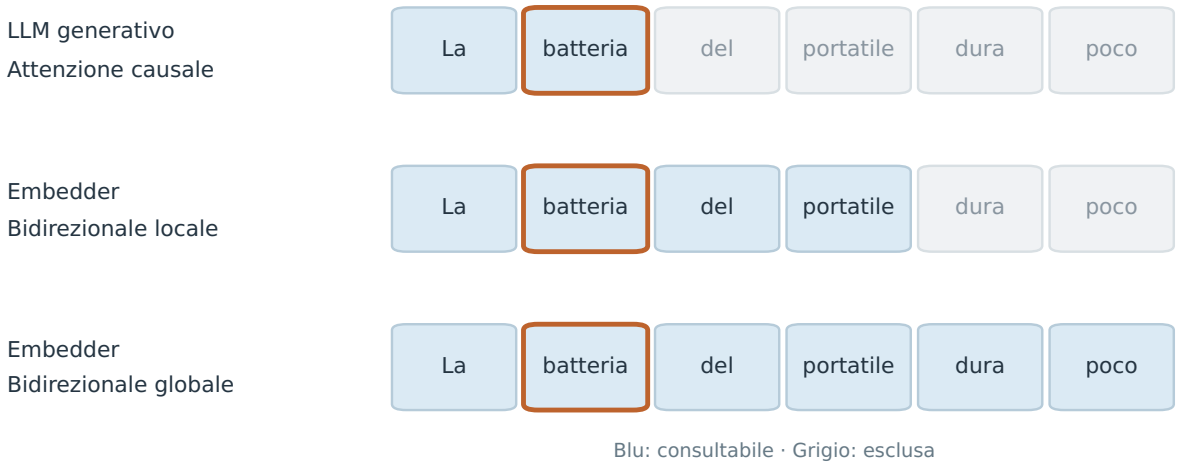


Figura 6. Aggiornamento della posizione «batteria», in arancione. Le posizioni blu possono contribuire, quelle grigie sono escluse. La finestra locale di due posizioni è illustrativa; il modello lavora su token.

L'attenzione globale richiede più calcolo su sequenze lunghe. EmbeddingGemma 2 alterna cinque strati locali e uno globale, per un totale di 24 strati. Gli strati globali permettono di collegare anche elementi lontani nella sequenza, come una didascalia e un'immagine. La maschera di attenzione assegna peso zero ai collegamenti esclusi. (configurazione del modello)

6.7 Dalla sequenza a un vettore

All'uscita del backbone resta un vettore per ogni posizione. Per ottenere **un embedding di dimensione fissa per l'intero input**, EmbeddingGemma 2 applica una proiezione, il mean pooling e la normalizzazione L2.

1. Proiezione a 768 dimensioni. Il vettore h_i rappresenta la posizione i dopo la normalizzazione interna finale del Transformer e contiene 512 numeri. Una trasformazione appresa lo porta a 768 dimensioni:

$$\mathbf{r}_i = W\mathbf{h}_i, \qquad W \in \mathbb{R}^{768 \times 512}.$$

Come si legge. La matrice W contiene pesi appresi. Moltiplicandola per h_i si ottiene r_i: ciascuna delle sue 768 coordinate è una combinazione delle 512 coordinate in ingresso. Rimane un vettore per posizione. (codice Transformers)

2. Mean pooling. Si calcola la media dei vettori delle posizioni valide, coordinata per coordinata. Il risultato è un unico vettore di 768 numeri, indipendentemente dalla lunghezza della sequenza:

$$\mathbf{u} = \frac{\sum_i m_i \mathbf{r}_i}{\sum_i m_i}.$$

Come si legge. m_i vale 1 per una posizione da includere e 0 per una da escludere. Al numeratore si sommano i vettori inclusi; al denominatore si contano le posizioni valide. Con tre posizioni valide si sommano quindi i tre vettori e si divide ogni coordinata per tre. È il pooling usato dal pacchetto ufficiale Sentence Transformers. (configurazione Sentence Transformers)

Sono escluse, in particolare, le posizioni di *padding*, aggiunte per elaborare insieme sequenze di lunghezze diverse. Includerle farebbe dipendere la media anche dalla quantità di padding. I vettori validi contengono già le informazioni su ordine e contesto elaborate dal Transformer.

3. Normalizzazione L2. Il vettore non nullo viene diviso per la sua norma L2. La lunghezza geometrica diventa uno, mentre la direzione e il numero di coordinate restano invariati:

$$\mathbf{z} = \frac{\mathbf{u}}{\|\mathbf{u}\|_2}.$$

Come si legge. Si divide ogni coordinata di u per $\|u\|_2$, la sua norma L2. Con due coordinate, (3; 4) ha norma 5 e diventa (0,6; 0,8). Il vettore z ottenuto ha norma uno. È l'ultimo passaggio della configurazione ufficiale. (configurazione Sentence Transformers)

Fra vettori normalizzati in questo modo, il prodotto scalare coincide con la similarità coseno. Per calcolarlo basta moltiplicare le coordinate corrispondenti dei due vettori e sommare i prodotti.

7. Contesto, chunking e Matryoshka

Per costruire un archivio bisogna scegliere che cosa rappresenta ogni embedding: un file intero, una pagina, un paragrafo o un intervallo audio. Questa scelta influisce sui risultati della ricerca e sulla dimensione dell'indice.

La **finestra di contesto** limita quanto contenuto si può elaborare in una chiamata; il **chunking** suddivide il contenuto in parti da indicizzare; la **dimensione dell'embedding** determina quanti numeri si conservano per ciascuna parte.

7.1 La finestra di contesto

La sequenza condivisa del capitolo 6 ha una capacità massima: la *finestra di contesto*. In EmbeddingGemma 2 contiene fino a 8.192 token complessivi. Le modalità condividono questo budget. La model card riporta, nelle impostazioni predefinite, 280 token per immagine, 140 per frame video e 25 per secondo di audio. (model card)

Input	Consumo indicativo	Conseguenza
Testo	Un token per unità prodotta dal tokenizer	Le parole non danno un conteggio esatto
Immagine	280 token con il budget predefinito	Più immagini riducono lo spazio per il resto
Video	140 token per frame con le impostazioni predefinite	Il costo dipende dai frame effettivamente selezionati
Audio	25 token al secondo	Circa 327 secondi come stima per il solo audio

Marker, timestamp e prefissi possono occupare altri token, e il processor può imporre limiti aggiuntivi. Per esempio, $8.192/140 \approx 58$ frame è un limite teorico, mentre il processor può selezionarne meno. La durata di video rappresentabile dipende quindi dal campionamento dei frame e dall'eventuale elaborazione dell'audio.

Con 4.800 token di testo, quattro immagini da 280 token e dieci secondi di audio, il conto è $4.800 + 4 \times 280 + 10 \times 25 = 6.170$ token. Restano 2.022 token per marker, timestamp e altri elementi. Se il testo occupa già 8.000 token, invece, bisogna accorciarlo o dividerlo prima di aggiungere gli stessi media.



Figura 7. Esempio di occupazione del contesto. Il margine deve contenere anche marker, timestamp e gli altri elementi aggiunti dal processor.

7.2 Chunking: dividere i contenuti

Un contenuto che supera la finestra di contesto deve essere diviso. Il chunking può però essere utile anche per input più brevi: un unico embedding di un manuale riassume insieme tutte le procedure, mentre chi cerca come scollegare una batteria ha bisogno del passaggio specifico.

Ogni parte, o *chunk*, può essere una sezione del manuale, una pagina o un intervallo di una registrazione. Si calcola un embedding per chunk e lo si salva insieme al riferimento alla posizione nel file originale.

Prendiamo una registrazione audio di 90 secondi su un intervento di manutenzione. Con il conteggio indicativo di 25 token al secondo occuperebbe circa 2.250 token: entra nel contesto, ma un embedding unico non ci direbbe dove viene descritto lo scollegamento della batteria. Possiamo invece creare quattro estratti di 30 secondi, facendoli iniziare ogni 20 secondi, e calcolare un vettore per ciascuno.

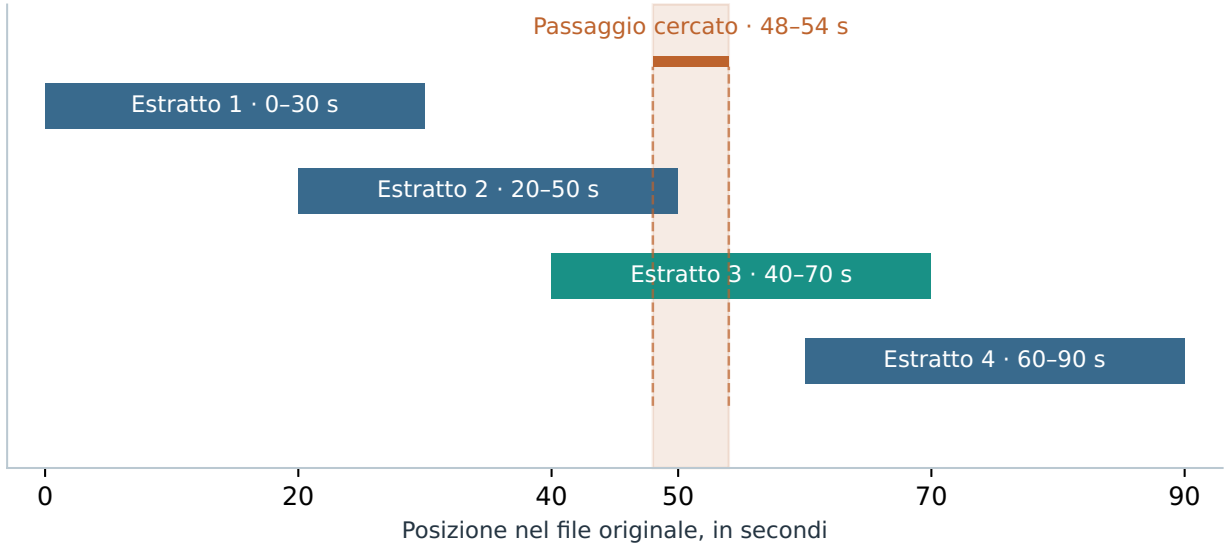


Figura 8. L'episodio fra i secondi 48 e 54 attraversa il confine del secondo chunk, ma rientra nel terzo (40–70 secondi). L'embedding identifica il chunk; i confini esatti dell'episodio sono indicati solo per spiegare l'esempio.

Se la ricerca seleziona il terzo chunk, il programma può aprire la registrazione al secondo 40. Il punto esatto in cui comincia l'episodio va poi individuato dentro quell'intervallo. Per un manuale, lo stesso riferimento permette di aprire la pagina o la sezione trovata.

L'*overlap*, cioè la sovrapposizione fra chunk consecutivi, riduce il rischio di spezzare un passaggio senza conservarlo per intero in nessun chunk. Chunk troppo brevi perdono contesto: «scollegare questo cavo» è poco utile senza sapere quale. Chunk troppo lunghi mescolano argomenti. La dimensione va scelta in base ai contenuti e alle domande a cui la ricerca deve rispondere.

Il chunking aumenta il numero di vettori da conservare. L'overlap può inoltre produrre risultati ripetuti, da raggruppare prima di mostrarli. Su archivi grandi, anche la dimensione di ciascun embedding incide sulla memoria e sui tempi di confronto.

7.3 Matryoshka: ridurre lo spazio per ogni vettore

Con milioni di chunk, conservare e confrontare 768 numeri per embedding richiede molta memoria e calcolo. Vettori più corti riducono entrambi, purché conservino informazioni utili alla ricerca.

Il **Matryoshka Representation Learning** (MRL) addestra il modello a produrre embedding utilizzabili anche prendendone solo le prime coordinate. Il nome richiama le bambole russe una dentro l'altra. EmbeddingGemma 2 supporta vettori di 768, 512, 256 e 128 dimensioni. (articolo MRL; guida all'inferenza)

Durante l'addestramento, lo stesso compito viene valutato con il vettore completo e con le prime 128, 256 e 512 coordinate. Le loss contribuiscono insieme all'aggiornamento dei pesi: le prime coordinate imparano a rappresentare il contenuto da sole; le successive possono aggiungere dettaglio.

Le prime m coordinate formano il *prefisso* del vettore. Per passare a 256 dimensioni si conservano le prime 256 coordinate e si applica di nuovo la normalizzazione L2:

$$\mathbf{z}^{(m)} = \frac{\mathbf{z}_{1:m}}{\|\mathbf{z}_{1:m}\|_2}.$$

Come si legge. $z_{1:m}$ contiene le prime m coordinate di z. Si divide questo vettore per la sua norma L2, ottenendo $z^{(m)}$. La sua dimensione è m, per esempio 256, e la sua norma è uno.

Dopo il taglio, la norma può essere inferiore a uno. La normalizzazione permette di usare ancora il prodotto scalare come similarità coseno. La stessa dimensione e la stessa normalizzazione vanno applicate sia agli embedding dei documenti sia a quelli delle query.

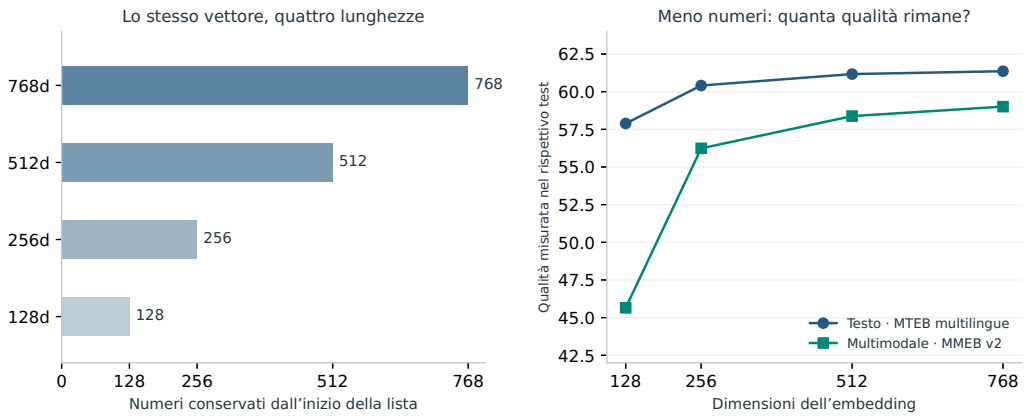


Figura 9. A sinistra, quattro prefissi dello stesso embedding. A destra, i risultati riportati da Google: diminuendo le dimensioni si risparmia spazio, ma i punteggi possono scendere. Le due curve riguardano benchmark diversi, uno testuale e uno multimodale. (model card)

La dimensione va scelta provando la ricerca sul proprio archivio. Vettori più corti riducono la memoria e accelerano i confronti, ma possono peggiorare i risultati. Tagliare un embedding già calcolato non riduce il lavoro del modello: il risparmio riguarda l'indice e la ricerca.